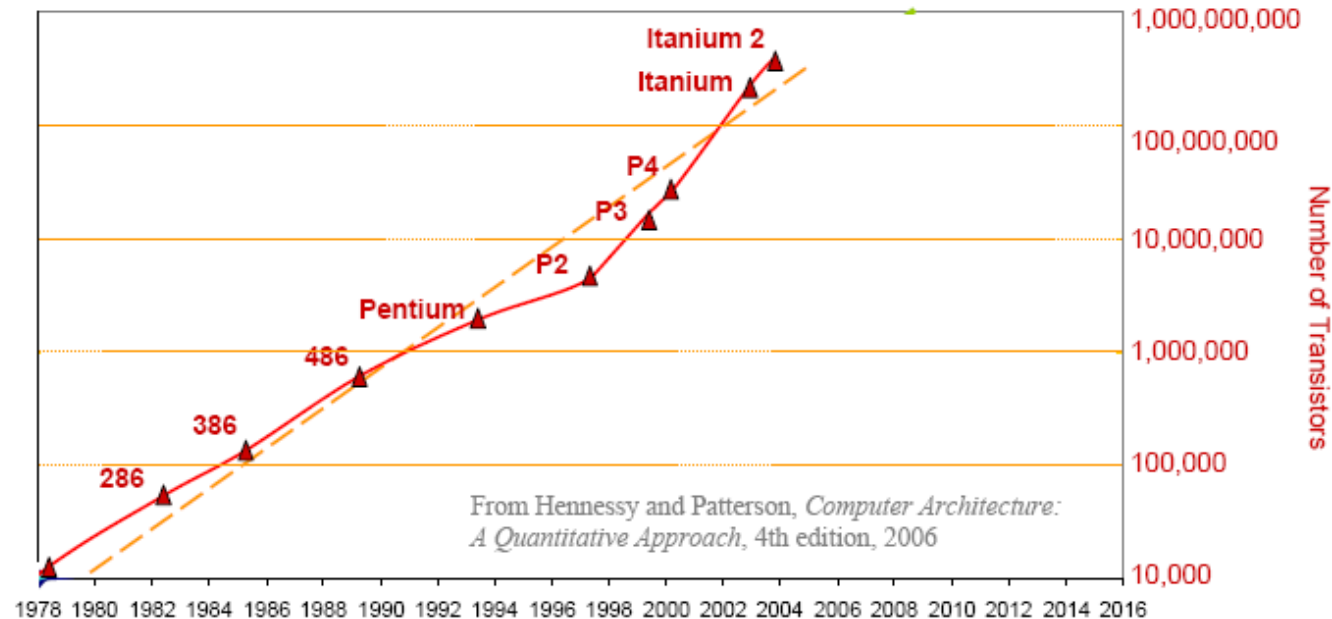


Multiprocessor and Mixed-Criticality Scheduling

Luís Miguel Pinho

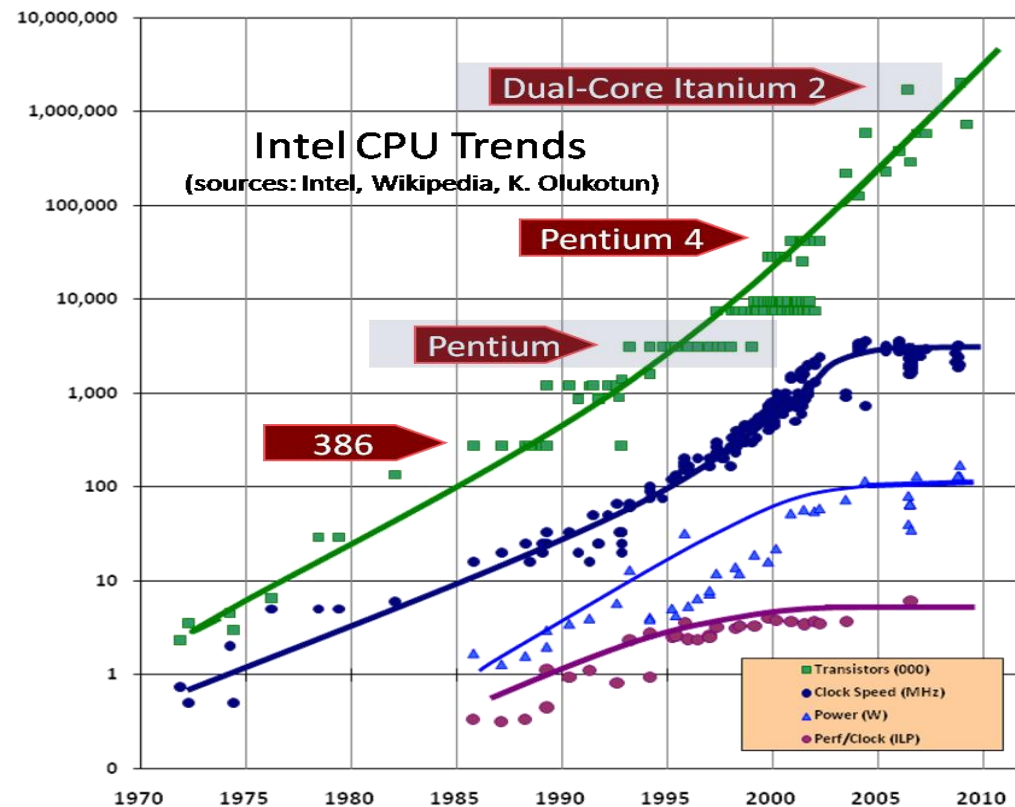
Importance of Parallelism

- ▶ Moore's Law: The quantity of transistors that can be placed on an integrated circuit has doubled approximately every two years (1965, revised 1975).



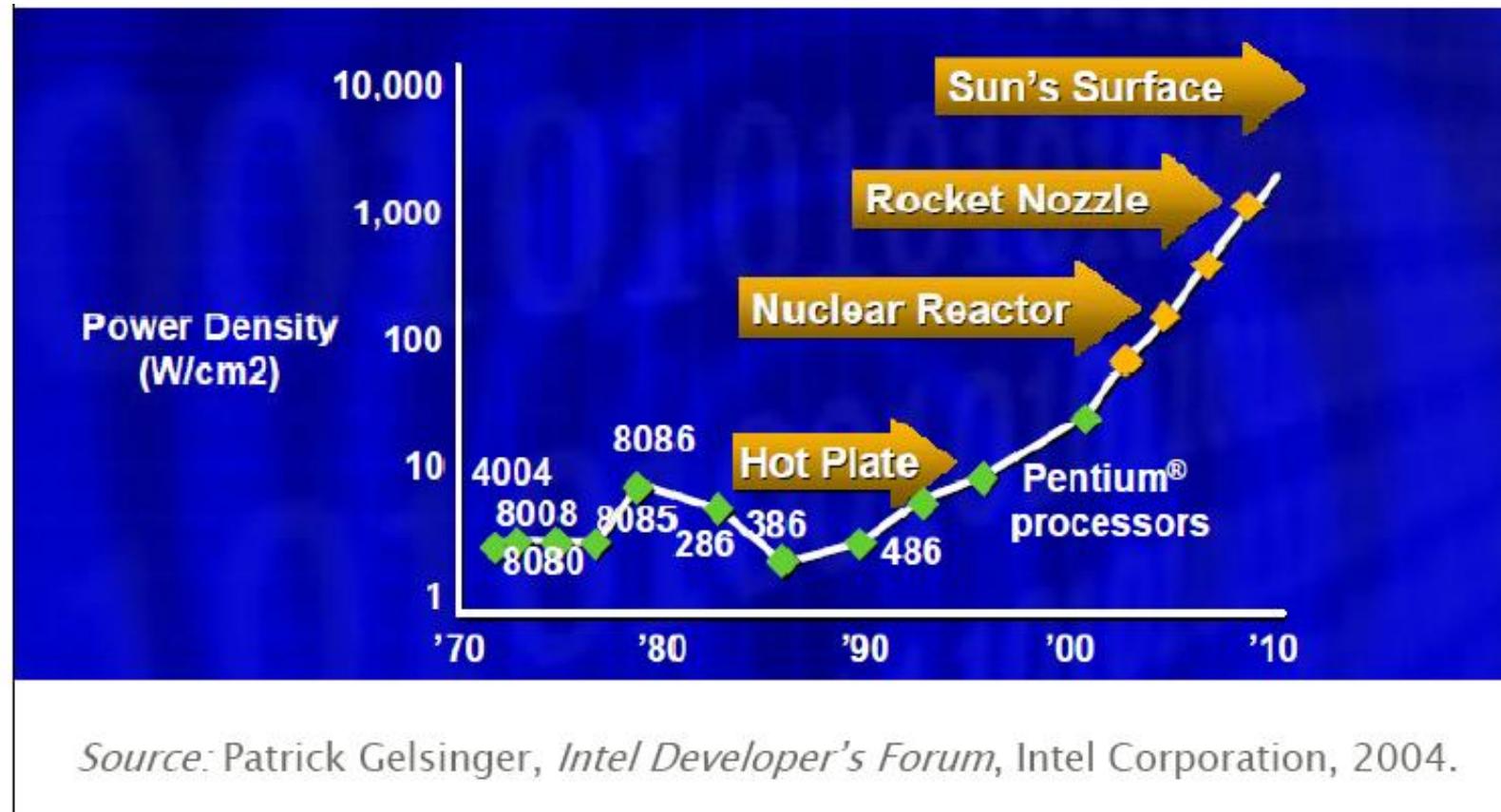
Importance of Parallelism

- ▶ At the same time processors were increasingly faster
 - ▶ Until ~2005



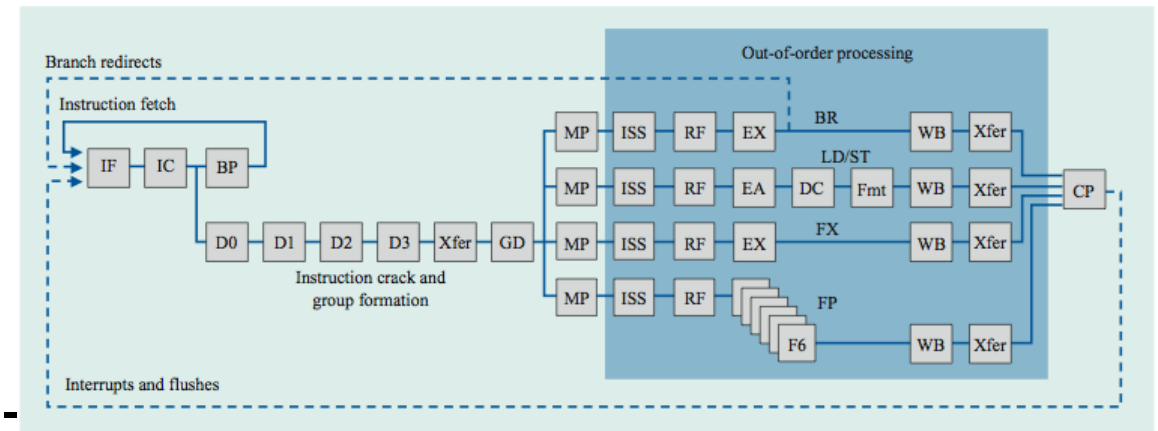
Importance of Parallelism

► The Power Wall



Importance of Parallelism

- ▶ **Memory wall**
 - ▶ Access times to memory are another wall
 - ▶ Memory technology is also changing
 - ▶ Fast is expensive
 - ▶ Power efficiency is also an issue
- ▶ **Instruction-level parallelism (ILP) wall**
 - ▶ In the '80s: expansion
 - ▶ Improvement in performance: 50%/year
 - ▶ Implicit parallelism (pipeline processor: 10 CPI -
 - ▶ The '90s: diminishing returns
 - ▶ Smaller improvement in implicit parallelism
 - out-of-order issue, branch prediction: 1 CPI -> 0.5 CPI
 - ▶ performance below expectations



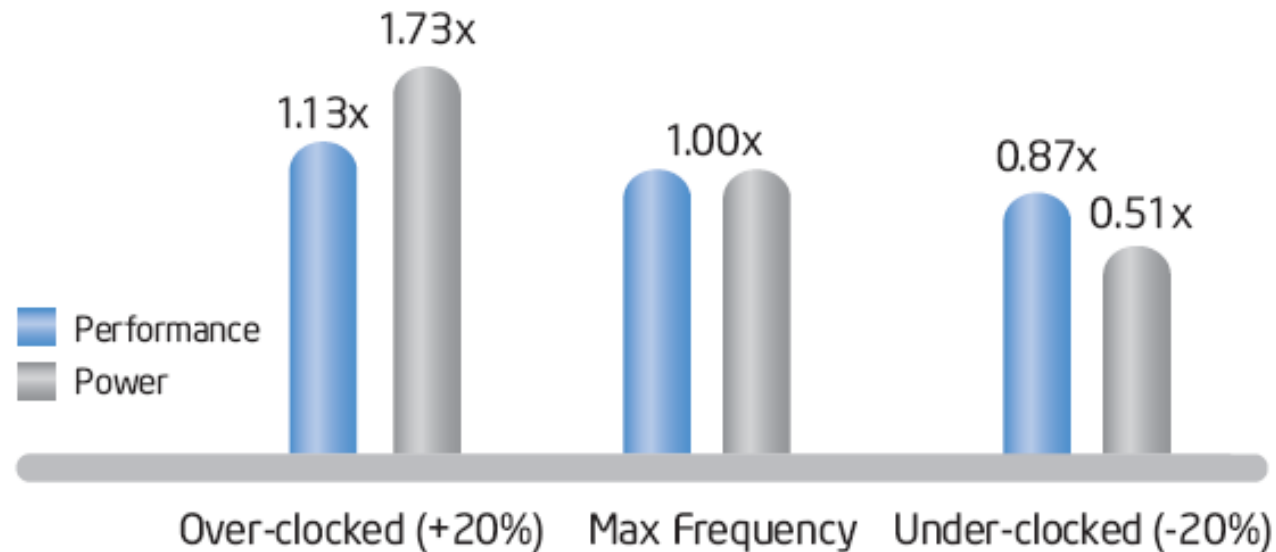
Importance of Parallelism

- ▶ **Power wall**
 - ▶ Cannot clock processors faster
- ▶ **Memory wall**
 - ▶ Performance is dominated by memory access times
- ▶ **Instruction-level Parallelism (ILP) wall**
 - ▶ No more extra work to keep functional units busy

- ▶ **But the IT industry depends on increased performance**
 - ▶ Software developers also
 - ▶ Solution was to parallelize CPUs

Importance of Parallelism

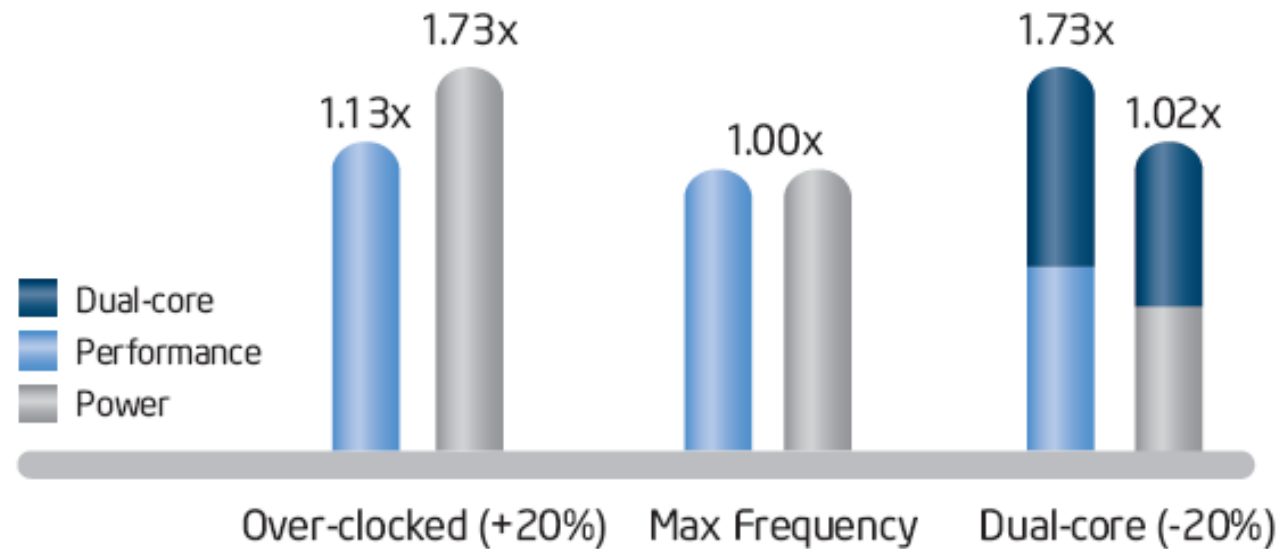
► Relation between speed and performance



Source: Intel® Multi-Core Processors - Making the Move to Quad-Core and Beyond

Importance of Parallelism

► Relation between speed and performance



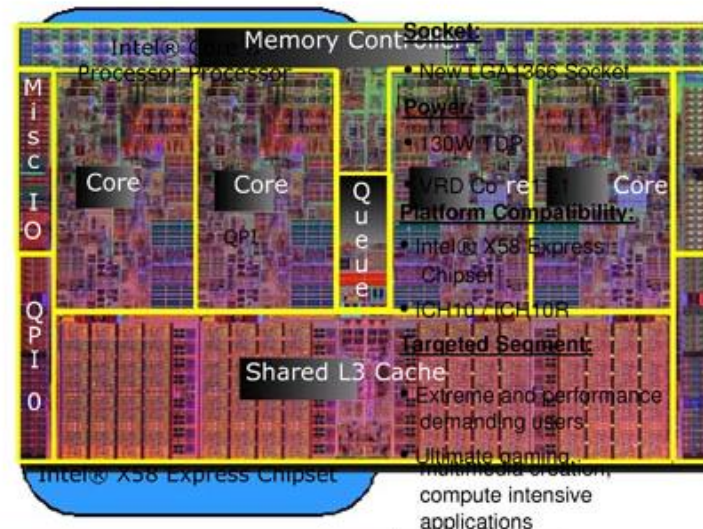
Source: Intel® Multi-Core Processors - Making the Move to Quad-Core and Beyond

Importance of Parallelism

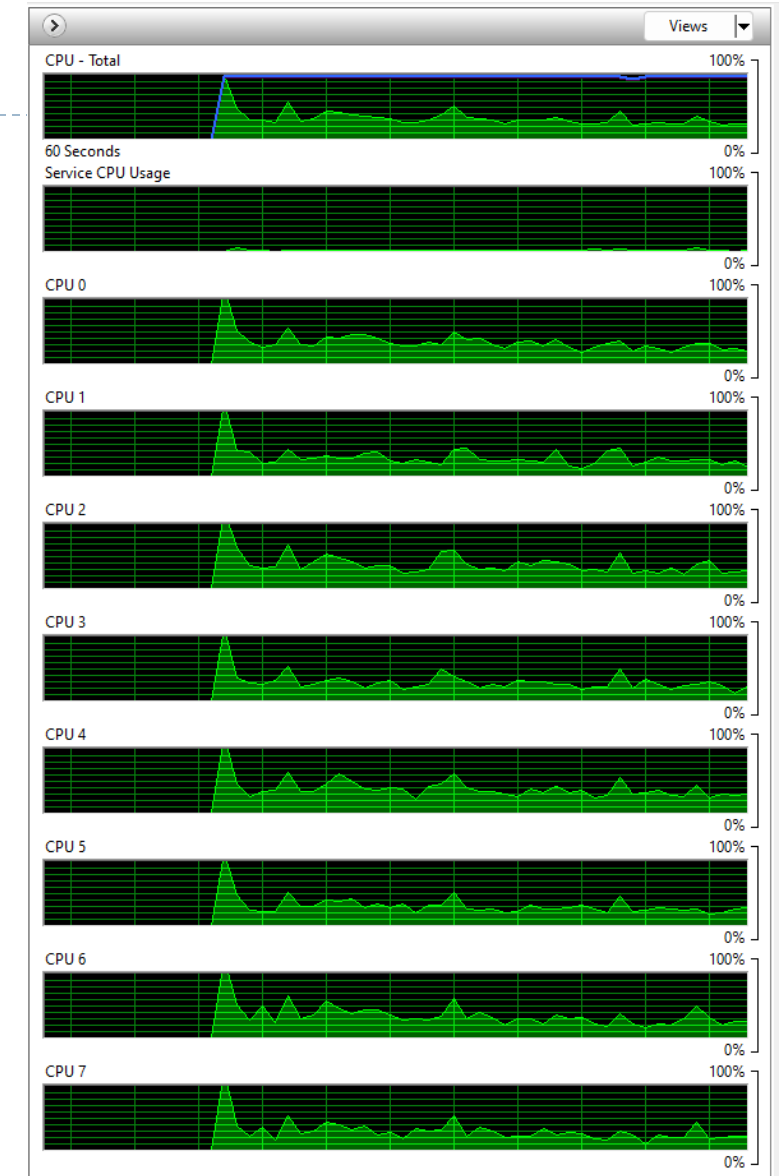
► Intel Core i7

Performance/Features:

- 8 processing threads via Intel® Hyper-Threading Technology (HT)
- 4 cores
- Turbo Mode operation
- Intel® QuickPath Interconnect (Intel® QPI) to Intel® X58 Express Chipset
- Integrated Memory Controller (IMC) - 3ch DDR3
- 7 more SSE4 instructions
- Overspeed Protection Removed



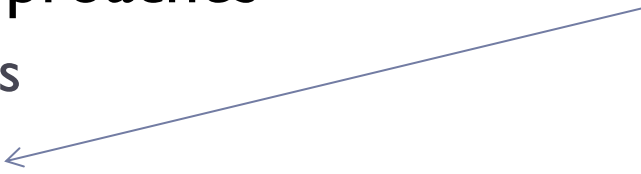
Intel's Next Gen Computing Genius!



Importance of Parallelism

- ▶ During decades, programmers had the processing power for free
 - ▶ HW designers had to cope with increasing requirements for processing
 - ▶ SW engineers using higher-level concurrency and parallel oblivion tools
- ▶ 2005: The Free Lunch Is Over
 - ▶ A Fundamental Turn Toward Concurrency in Software, Herb Sutter, Microsoft
- ▶ HW becomes flexible
 - ▶ HW designers can focus on parallelizing simpler cores
 - ▶ And SW engineers need to adapt

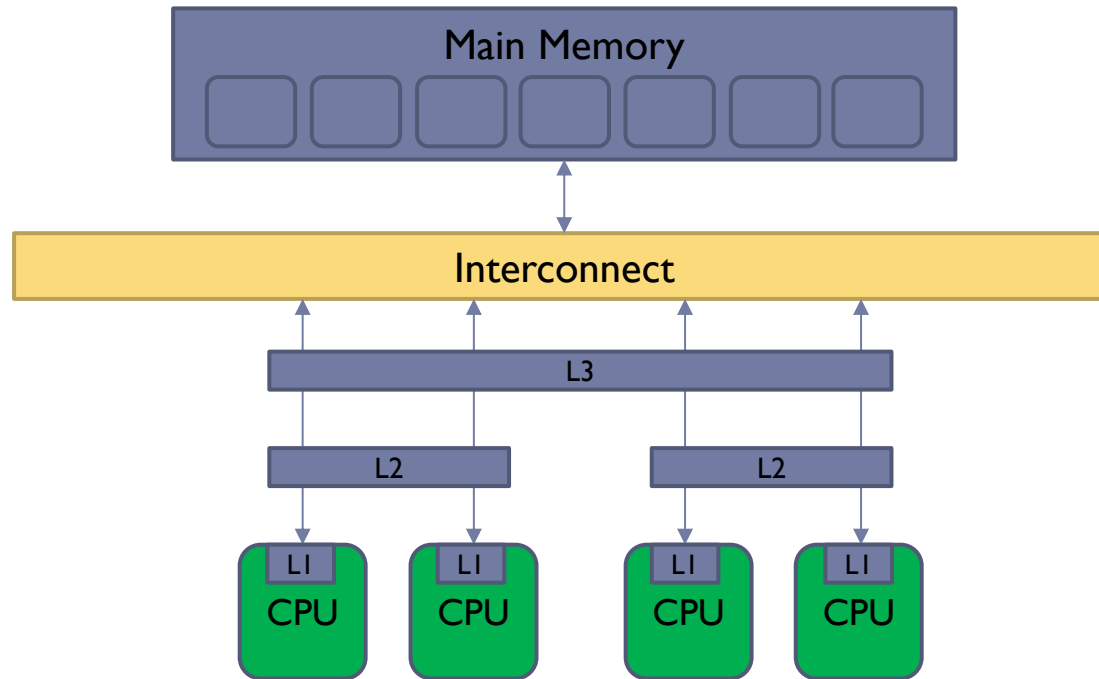
Importance of Parallelism

- ▶ **Implicit approaches**
 - ▶ ILP, OoO, TLP
 - ▶ Hyperthreading
 - ▶ Parallelizing compiler
- ▶ **Explicit approaches**
 - ▶ Processes
 - ▶ Threads 
 - ▶ Light-weight threads (tasks)
- ▶ **Implicit approaches**
 - ▶ Specify what to parallelize, not how (if possible)

Executing in multiple
cores

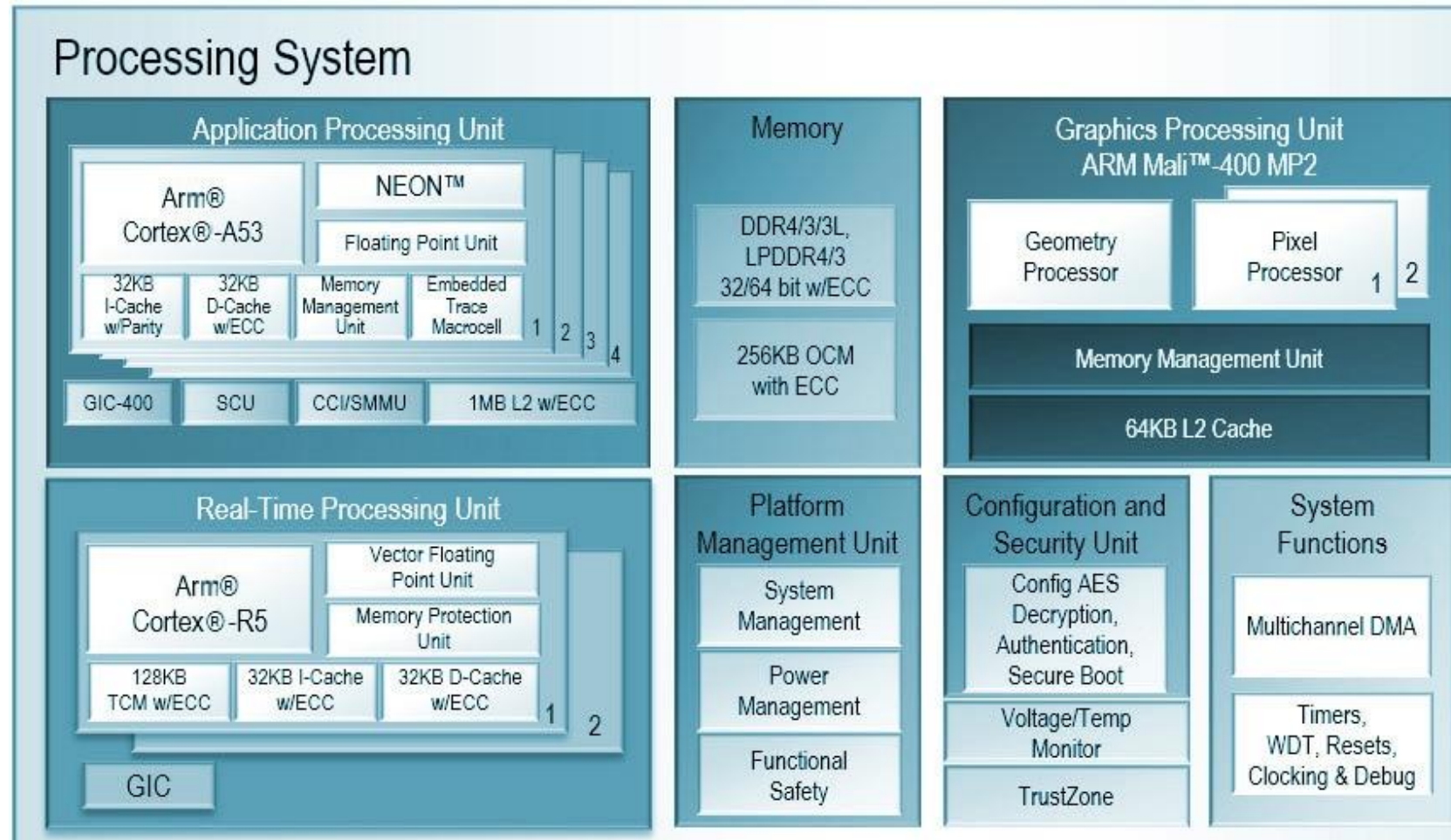
Shared Memory Systems

- ▶ First, shared memory systems
 - ▶ All cores can access the same physical memory of the system



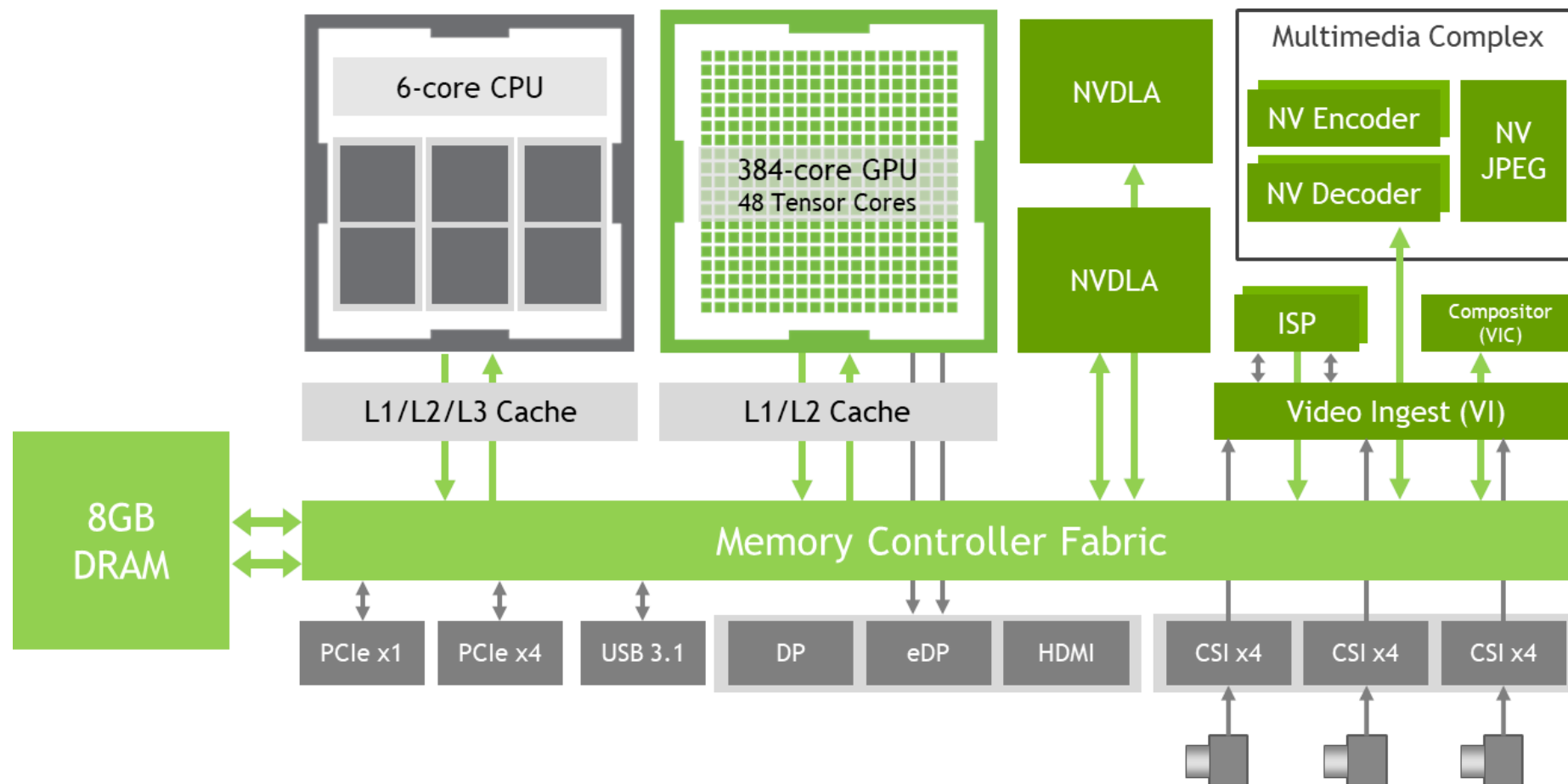
Heterogenous Systems

► Getting more complex



Heterogenous Systems

► Getting more complex



Real-time Systems Challenges

- ▶ How to partition the application
 - ▶ Coupling and Cohesion
- ▶ How to map the different components to different cores (and accelerators)
 - ▶ Potentially NP-hard
- ▶ How to schedule the components
 - ▶ Global, partitioned, clustered
- ▶ Access to HW resources
 - ▶ Shared memory, bus
 - ▶ (Distributed) Blocking, contention, extreme worst-case

Real-time Systems Challenges

► Predictability and Performance

- The performance of an application can vary significantly depending on how tasks are allocated and scheduled on a multicore platform
 - How to allocate and schedule concurrent tasks in a multicore platform?
 - How to analyze real-time applications to guarantee timing constraints while accounting for communication delays and interference?
 - How to reduce interference?
 - How to optimize resource allocation?
- Assuming intra-task parallelism exists, how to deal with it?
 - Next year

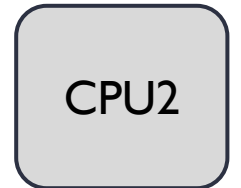
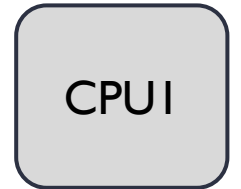
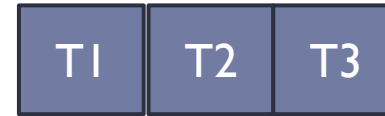
Real-time Systems Multiprocessor models

- ▶ Identical: Each processor in the platform has the same computing capabilities as every other processor.
 - ▶ Symmetric Multiprocessor systems
- ▶ Uniform (or related): Processors are similar, except for computing capacity s . Thus, a job that executes on a processor of computing capacity s for t time units completes $s * t$ units of execution
 - ▶ Same processor at different speeds
- ▶ Unrelated: Each job may have a different execution rate on each processor. Thus, if a job has an execution rate r specified for a processor then executing it on that processor for t time units completes $r * t$ units of execution.
 - ▶ Heterogeneous processors

Real-time Systems Scheduling models

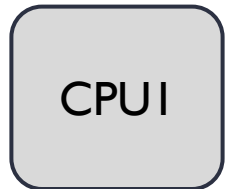
▶ Global scheduling

- ▶ Allows any job to execute on any given processor
- ▶ Job migrations are allowed



▶ Partitioned Scheduling

- ▶ A task is mapped to a particular processor and it is only allowed to execute its jobs on the processor to which it has been mapped
- ▶ Job migrations are not allowed



Real-time Systems Scheduling models

▶ Global scheduling

- ▶ Allows in theory for better utilization and overall system performance
 - ▶ Load automatically balanced
 - ▶ Easier re-scheduling (dynamic loads, selective shutdown, etc.)
 - ▶ Lower average response time (see queueing theory)
 - ▶ More efficient reclaiming and overload management
 - ▶ Number of preemptions

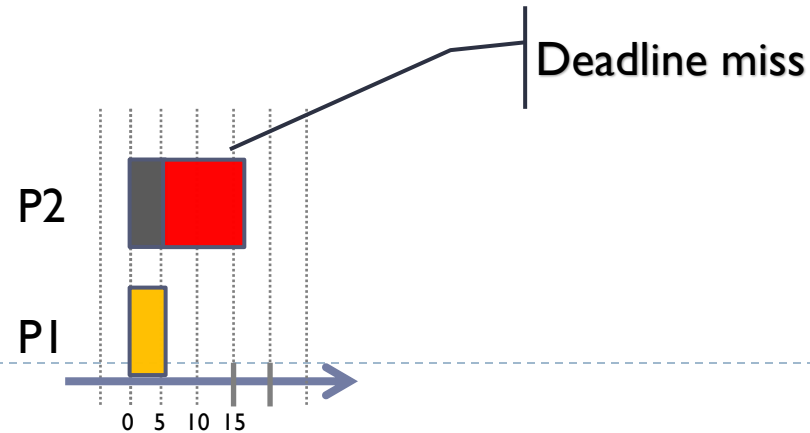
▶ But

- ▶ Migration overhead: can be mitigated by proper HW (e.g., cache snooping mechanisms)
- ▶ Schedulability gap, only sufficient tests

Real-time Systems Scheduling models

▶ Global Scheduling

- ▶ Optimal schedulers only for implicit deadlines:
 - ▶ $U_{tot} \leq m$
 - ▶ Pfair, EKG, LLREF, DP-fair, RUN, etc.
 - ▶ Preemption and synchronization issues
- ▶ Scheduling overhead (migration introduces overheads and potential conflicts)
- ▶ No optimal scheduler known for more general task models
- ▶ Classic schedulers are not optimal: Dhall's Effect
 - ▶ Quick exemple
 - ▶ 3 tasks: (5, 15), (5, 15), (12, 16)
 - ▶ Utilization $U = 142\%$
 - ▶ 2 processors (200% capacity)



Real-time Systems Scheduling models

▶ Global Scheduling

- ▶ There are more schedulability tests than researchers
 - ▶ Many tests for specific types of tasksets or tailored schedulers
 - ▶ No optimal scheduler is known for constrained or arbitrary deadline systems
- ▶ For implicit deadline systems ($D_i = T_i$), for EDF
 - ▶ Utilization-based test (tight)

$$\sum_{\tau_i \in \tau} U_i \leq m(1 - U_{max}) + U_{max}$$

$$U_{max} = \max \left\{ \frac{C_i}{T_i} \right\}$$

Real-time Systems Scheduling models

- ▶ With partitioned scheduling, each processor can be treated independent and all theory of single processor scheduling still applies (to each processor)
 - ▶ Assigning tasks to processors is the difficult (and inefficient) issue to guarantee
 - ▶ No processor is over-utilized
 - ▶ Each partition is schedulable
- ▶ Bin packing problem
 - ▶ Given a number of bins B , a bin capacity V , and a set of items $t_1, \dots, t_n$ with sizes $c_1, \dots, c_n$, does there exist a packing of $t_1, \dots, t_n$ that fits into B bins?
 - ▶ Given a bin capacity V and a set of items $t_1, \dots, t_n$ with sizes $c_1, \dots, c_n$, assign each item to a bin such that the number of bins is minimized

Real-time Systems Scheduling models

▶ Bin packing

- ▶ NP hard problem, so usually heuristics are used
 - ▶ First-fit (FF): The processors are considered ordered in some manner and the task is assigned to the first processor on which it fits
 - ▶ Worst-fit (WF): A given task is assigned to the processor with the maximum remaining capacity
 - ▶ Best-fit (BF): A given task is assigned to the processor with the minimum remaining capacity (but still has enough capacity to fit its weight)
 - ▶ Tasks can be ordered in an increasing or decreasing order, e.g.:
 - First-fit decreasing: Start with the tasks with higher utilization and assign to the first processor it fits
 - Best-fit increasing: Start with the tasks with lower utilization and assign to the processor with minimum remaining capacity
 - Etc...
 - ▶ And many others

Real-time Systems Scheduling models

► Bin packing

► Example: 7 loads

► $l_1 = 10$

► $l_2 = 2$

► $l_3 = 14$

► $l_4 = 1$

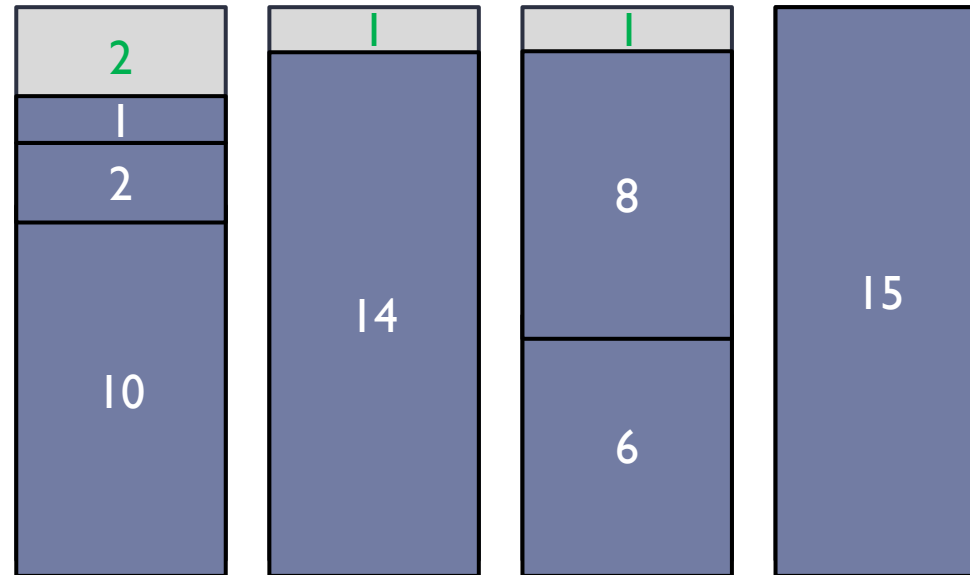
► $l_5 = 6$

► $l_6 = 15$

► $l_7 = 8$

► Bin capacity = 15

First-fit



Real-time Systems Scheduling models

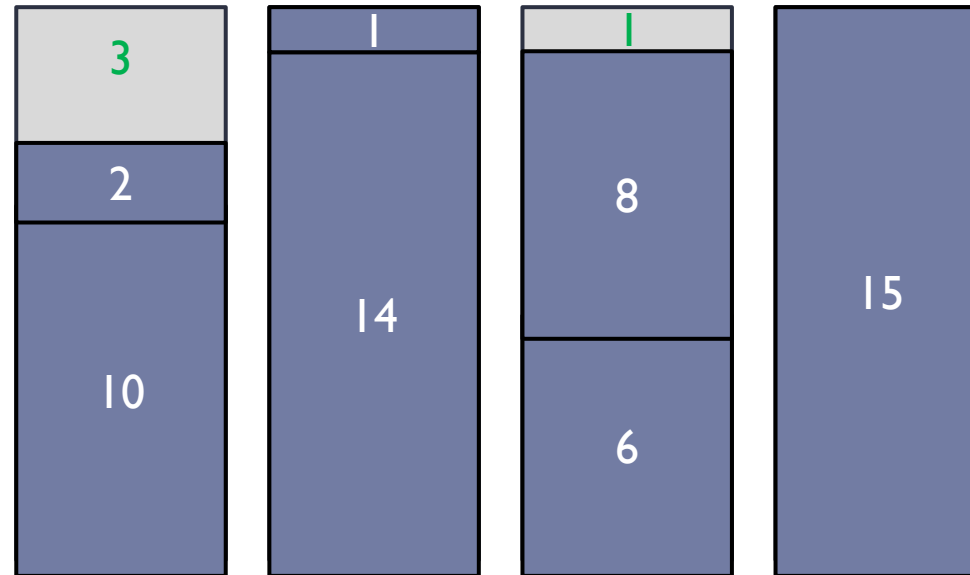
► Bin packing

► Example: 7 loads

- $L1 = 10$
- $L2 = 2$
- $L3 = 14$
- $L4 = 1$
- $L5 = 6$
- $L6 = 15$
- $L7 = 8$

► Bin capacity = 15

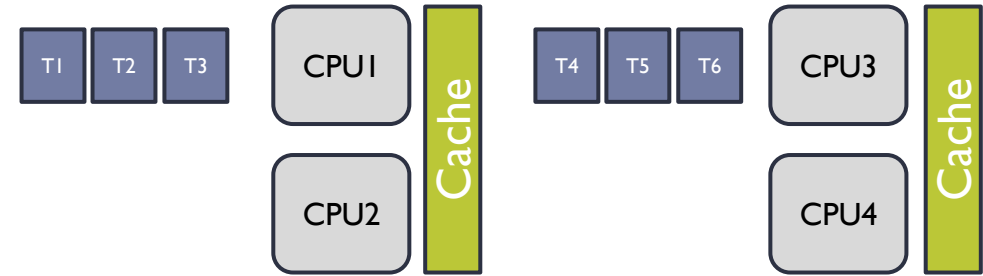
Best-fit



Real-time Systems Scheduling models

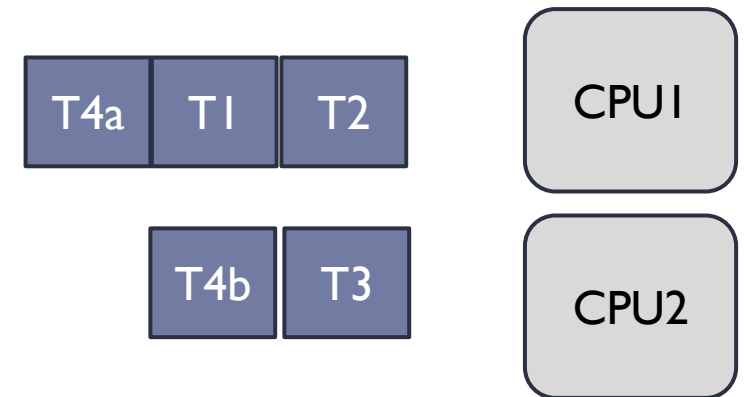
▶ Clustered Scheduling

- ▶ The processors of a given platform are partitioned in clusters and each task is mapped to one of those clusters
- ▶ Job migrations are allowed within a cluster



▶ Semi-partitioned

- ▶ A few tasks are allowed to migrate while others are mapped to processors (not allowed to migrate)
- ▶ E.g. task-splitting



Real-time Systems Scheduling models

► Exercise

- Trace the execution of the following task set with 2 processors for 30 ms

	Arrival (ms)	C (ms)	T=D (ms)
τ_1	2	5	10
τ_2	3	2	9
τ_3	2	1	5
τ_4	3	3	9
τ_5	0	7	12

- Using Global EDF and Global RM
- Using Partitioned EDF and Partitioned RM
 - Try First Fit or Best-fit decreasing

Resource sharing

- ▶ In multiprocessor systems, data may be shared by tasks executing in different cores
- ▶ Necessary to have mechanisms to synchronize access to the shared data
 - ▶ Extensions of existing approaches, e.g.
 - ▶ Multiprocessor Priority Ceiling Protocol (MPCP)
 - ▶ Multiprocessor Stack Resource Policy (MSRP)
 - ▶ Or new protocols, e.g.
 - ▶ Multiprocessor resource sharing Protocol (MrsP)
 - Use of spinning and helping mechanisms, supports partitioned and global
- ▶ Goal is to maintain as much as possible the properties of bounded priority inversion and free of deadlocks

Impact of heterogeneous platforms

- ▶ Execution of accelerated code in real-time systems is addressed as blocking
 - ▶ Tasks in the CPU are blocked during the execution in the accelerator
 - ▶ The analysis may consider that during blocking tasks are spinning waiting for the result
 - ▶ Or suspended (less pessimistic but much more complex analysis)
 - ▶ Access to accelerators may be provided by server tasks, which improve analysability
- ▶ Execution time in accelerators is a complex issue
 - ▶ E.g. WCET in GPUs is impossible to determine through static methods due to complexity of design, and lack of details of specification

Timing and Schedulability Analysis

- ▶ Parallel platforms and parallel applications introduce significant challenges to the analyses that guarantee correct execution
 - ▶ Finding worst-case execution time is extremely difficult
 - ▶ Contention of parallel execution when accessing hardware shared resources (e.g., memory)
 - ▶ Even when executing in different processors
 - ▶ Finding worst-case response time is increasingly complex (and pessimistic)
 - ▶ Needs to cope not only with interference from other activities
 - ▶ As well as interference from parallel segments of the same activity

Timing and Schedulability Analysis

- ▶ Worst-case scenarios take into account corner cases, and need to consider all potential interactions
 - ▶ This may cause a significant inflation of the execution and response times
 - ▶ Or be intractable in practice
 - ▶ Several analyses give response times larger than the sum of all execution times, even on a single activity
 - ▶ Predictability is higher if some level of “*staticness*” is provided to the system
 - ▶ Partitioned scheduling

Mixed-Criticality Systems

- ▶ In these systems, tasks of different criticality share the processor (reduce cost)
 - ▶ Using the same scheduler
- ▶ Mode-based scheduling is used to adapt execution properties
 - ▶ Maximize utilization, and
 - ▶ Guarantee schedulability of critical functionality
- ▶ Priority-based scheduling can be used, possibly with the support of servers
- ▶ The largest majority of studies disregard the non temporal impact of mixed criticality
 - ▶ In most cases hypervisors need to be used, with independent schedulers for different critical functions

Mixed-Criticality Systems

- ▶ In the original model (Vestas model), for MCS, tasks are characterized by:
 - ▶ Period, Deadline, Arrival
 - ▶ Criticality level
 - ▶ Set of worst-case execution time, one per level
- ▶ The idea is that in higher critical levels, more conservative analysis of worst-case execution time is used, so values of C may be higher, and highly pessimistic
- ▶ A simplification of the model is to consider only two criticality levels and two execution times

Mixed-Criticality Systems

- ▶ The model was later extended to the notion of modes of execution (Adaptive Mixed Criticality)
 - ▶ During execution, the system uses the more “optimistic” best-case execution times of lower criticality levels
 - ▶ If one the tasks overruns its best-case execution time, a mode change is triggered
 - ▶ Criticality level is increased
 - ▶ Tasks with level below the current level are dropped
 - ▶ Scheduling now considers the worst-case execution time values